## Senior Project 2

# RESEARCH PROPOSAL

| PROJECT TITLE |
| --- |
| **Mini-NPU: A CoralNPU-Inspired RISC-V Accelerator with a Decoupled int8 MAC Engine and Module-Level Clock Gating, Verified with a Self-Built UVM Environment** |

| Field | Information |
| --- | --- |
| Student / Team | Nguyễn Hoàng Công Tấn / Nguyễn Thế Bảo |
| Student ID(s) | 23161064 / 23144002 |
| Program / Cohort | Electronics and Telecommunications Engineering Technology, class 23161FIE |
| Supervisor | Van-Khoa Pham (Dr.) |
| Research Area | Edge-AI accelerators (RISC-V NPU), RTL design and UVM verification, low-power digital design |
| Project Duration | 16 weeks (12 Oct 2026 – 31 Jan 2027) |
| Submission Date | 11/11/2026 |

## Required Research Logic

| Stage | Core Question | Required Evidence |
|---|---|---|
| 1. Technical Issues | What exactly is technically wrong, limited, costly, slow, inaccurate, unsafe, or unsolved? | Quantified problem, constraints, metrics, research questions |
| 2. Related Works | How have existing studies or systems addressed the problem? | Critical comparison, results, limitations, research gap |
| 3. Proposed Idea | What new or adapted idea is proposed to close the gap? | System concept, architecture, novelty, hypotheses |
| 4. Methodology | How will the idea be implemented and objectively evaluated? | Implementation plan, baselines, experiments, metrics, analysis |
| 5. Expected Results | What measurable outcomes and deliverables are expected? | Target performance, contributions, prototype/report/demo |

## Recommended Writing Standards

- Use precise engineering terminology and measurable performance indicators rather than general claims such as “better,” “faster,” or “more efficient.”
- Use peer-reviewed and technically credible sources. Prefer recent work for the state of the art while retaining seminal references when necessary.
- For every important claim, provide evidence through citation, measurement, benchmark data, or a clearly stated assumption.
- Define all acronyms at first use and use consistent symbols, units, dataset names, hardware specifications, and evaluation conditions.
- The proposed solution must directly address the research gap identified from the related works; the evaluation metrics must directly reflect the technical issues defined in Section 1.

## Suggested Proposal Length

| Component | Suggested Length |
|---|---|
| Core Sections 1-5 | 6-10 pages |
| Figures / Tables | As required for technical clarity |
| References | Normally 10+ credible sources for a research-oriented project |
| Appendices | Optional; implementation details, extended tables, risk register, etc. |

# 1 TECHNICAL ISSUES / PROBLEM DESCRIPTION

*Define the engineering problem in measurable and technically defensible terms.*

### 1.1 Background and Technical Context

Machine-learning inference is moving from data centres to battery-powered, cost-constrained devices such as wearables, sensors, and smart-home endpoints. On these devices the workload is dominated by multiply-accumulate (MAC) operations on small integers, and post-training quantization to 8-bit integers (int8) preserves accuracy while allowing integer-only arithmetic [1]. Domain-specific accelerators exploit this regularity: the Tensor Processing Unit (TPU) uses a large 8-bit matrix unit [2], Eyeriss reduces data-movement energy with a

dedicated dataflow [3], and the survey in [4] shows that energy is governed by data reuse and by how well the arithmetic units are kept busy. The observation that arithmetic costs far less energy than moving data [5] makes small, tightly-coupled memories and wide operand ports attractive for an edge neural processing unit (NPU).

Open-source projects make such architectures studyable. Google's Coral NPU combines a small RISC-V scalar core with vector and matrix execution units and tightly-coupled memories [6]; Gemmini generates systolic-array accelerators attached to RISC-V cores [7]. This project follows the same idea at a scale that a two-person team can design, verify, and explain completely. A 5-stage RV32IM core (with the Zicsr, Zifencei, and Zbb extensions [8]) is extended with four custom instructions, LOAD_WEIGHT, LOAD_INPUT, COMPUTE, and READ_RESULT, that push commands through an 8-entry FIFO to a decoupled int8 outer-product MAC engine of N×M×K = 4×4×4 (16 processing elements, each a 4-way dot product, 64 MACs per cycle). Core and engine share a 64 KB dual-port data memory (DTCM). The target workload is a 784-64-32-10 int8 multilayer perceptron (MLP) on MNIST [9]. The topic belongs to Computer Engineering because it joins instruction-set extension, microarchitecture, memory-system design, bare-metal firmware, and hardware verification in one system, the hardware/software co-design theme of domain-specific computing [10].

In such a system the engine is idle whenever the core runs code that does not use it (address generation, loop control, activation, input/output). In a conventional RTL design the engine's clock still reaches every flip-flop of its operand buffers (2 × 256 × 128 bit), PE array, and accumulators in every cycle, so the clock network and the registers keep switching. Dynamic power grows with switching activity, capacitance, supply voltage squared, and frequency [11], and a free-running clock toggles in every cycle; clock gating is the standard remedy for idle registers [12][13][14]. Module-level gating with an enable derived from hardware state is simple in principle, but it creates two obligations that are the subject of this project. First, the gate must not change behaviour or timing, because any added or lost cycle breaks software that counts cycles. Second, its benefit must be quantified honestly: switching activity from RTL simulation is an estimate of power, not a measurement of it [15].

Industrial practice verifies such blocks with constrained-random Universal Verification Methodology (UVM) environments, golden reference models, assertions, and functional coverage [16][17]. This is rarely done at student scale, so the project builds that environment itself (three UVM environments for the engine, the core, and the whole SoC) and uses it as the evidence that the gated design is correct.

### 1.2 Technical Problem Statement

The technical problem addressed in this project is that a small RISC-V-attached int8 MAC engine wastes switching activity while it is idle, and that the cure, clock gating, can only be trusted if the gated design is proven to behave exactly like the ungated one. The ungated design (D0) clocks the whole engine datapath in every cycle regardless of its idle fraction; the gated design (D1) must remove that activity without adding a cycle, changing a result, or breaking an in-flight command. The project therefore has to (i) establish a verified, bit-exact functional baseline (D0) against independent golden models, (ii) prove D1 cycle- and result-identical to D0, and (iii) measure the reduction in switching activity as a function of the engine's idle ratio on synthetic and MNIST workloads.

### 1.3 Technical Constraints and Operating Conditions

| Constraint / Condition | Current or Required Value | Engineering Significance |
|---|---|---|
| Computing resources | QuestaSim 10.7c (UVM 1.2, DPI-C, coverage); Verilator 5.020 and RISC-V GNU toolchain under WSL2 (Ubuntu 24.04); Icarus Verilog 12 and GitHub Actions CI; Python 3 with numpy and PyTorch | QuestaSim runs the UVM flow and produces the activity dumps. Its feature set (randomize, covergroup, DPI-C) was checked in week 1. Verilator is used for lint only. |
| Memory / Storage | ITCM 16 KB; DTCM 64 KB (4 banks × 32 bit); WBUF and IBUF 256 × 128 bit each; command FIFO 8 entries; KMAX = 1024 elements; model data region 56,816 B | The whole MLP fits in the DTCM, so no DMA or external memory is needed. |

| Constraint / Condition | Current or Required Value | Engineering Significance |
|---|---|---|
| Latency / Real-time requirement | No real-time deadline. Engine commands take K/4 + 2 busy cycles (bound K/4 + 3). D1 must add 0 cycles to any program. | Cycle-identical D0/D1 behaviour (SC3) is the criterion that makes gating acceptable for software that reads mcycle. |
| Energy / Power | Switching-activity estimate from RTL simulation: toggles on clk_mac (M1) and on all nets of the engine (M2), per 1,000 cycles | Primary metric of the gating study. No synthesis or technology library is used, so results are never called “measured power”. |
| Accuracy / Quality | int8 MLP: 97.43 % on the 10,000 MNIST test images (float model 97.48 %); RTL logits bit-exact against the golden model on the first 1,000 images | Bit-exactness, not accuracy, is the success criterion; accuracy shows that quantization is not the limiting factor. |
| Communication / Bandwidth | DTCM port A 32 bit read/write (core); port B 128 bit read-only (engine), one quad of four words per cycle | Port B supplies one WBUF/IBUF entry per cycle, which keeps the 64-MAC PE array fed. |
| Cost / Area / Resource utilization | N×M×K = 4×4×4: 16 dot4 PEs (64 int8 multipliers); 8×8 only if time allows after the core criteria are met | Sets the engine size. Area is not evaluated because synthesis is outside the project. |
| Security / Privacy / Safety | Not a project objective. The engine drops illegal commands and records sticky error bits (err_k, err_addr, err_illegal, err_align). | Error handling is specified and verified, but no security analysis is made. |

### 1.4 Research Questions

**RQ1.** Does module-level clock gating driven by hardware-detected idleness leave the Mini-NPU cycle-identical and result-identical to the ungated design (D1 versus D0) on directed, random, synthetic, and MNIST workloads?

**RQ2.** How much does gating reduce the switching activity of the MAC engine (toggles on clk_mac and over all engine nets) as a function of the engine's idle ratio, and does the MNIST workload follow the trend predicted by synthetic workloads?

**RQ3.** Can a self-built UVM environment (constrained-random stimulus, an untimed C golden model over DPI-C, assertions, and functional coverage) close verification of the engine and of the whole SoC to the project's targets within the project duration?

### Success Criteria

| Criterion | Metric | Acceptance / Target Condition |
|---|---|---|
| SC1 Functional integrity | Scoreboard mismatches; assertion failures | Zero of both for every defined directed and random test, on D0 and D1 |
| SC2 Coverage | Functional coverage bins declared in the coverage plan | 100 % of declared bins on D0 before the gating work starts (week 10); every exclusion justified in writing |
| SC3 D0/D1 equivalence | SHA-256 of the retire log; final mcycle and minstret; output dump | Identical for the same program and seed. Fallback: result-identical, with the reason documented. |
| SC4 Switching activity | % change of M1 and M2, D1 versus D0 | Reported for idle ratios of 10–90 % (synthetic) and for MNIST; no target percentage is fixed in advance |
| SC5 MNIST | RTL logits and labels versus the golden model | Bit-exact on the first 1,000 test images (D0 and D1); the full 10,000-image run on D0 reproduces the golden accuracy |
| SC6 Reproducibility | Commands and recorded versions | One command per flow regenerates the regression, coverage, and activity data; tool versions and seeds are recorded |

# 2 EXISTING SOLUTIONS AND RELATED WORKS

*Establish the state of the art and derive a defensible research gap.*

## 2.1 Literature Scope and Search Strategy

The review covers four themes that map to the project's four contributions: (i) accelerator architectures and generators that couple a matrix engine to a host core; (ii) the energy cost of arithmetic, data movement, and idle hardware; (iii) clock gating and activity-based power estimation; and (iv) integer quantization and the verification methodology needed to prove an accelerator correct. Peer-reviewed work from 2014–2021 is used for accelerators and quantization, seminal papers from 1994–2000 are retained for clock gating and power estimation, and the open-source Coral NPU repository (commit 382b5c1) and its developer guides, read on 15 October 2026, represent the current open RISC-V NPU. Standards (IEEE 1800, 1800.2, and the RISC-V ISA manual) and two textbooks [11][18] define the terminology.

## 2.2 Related Work Summary

| Ref. | Problem / Context | Method / Architecture | Key Results | Limitations Relevant to This Project |
|---|---|---|---|---|
| [2] Jouppi et al. (2017) | Cost of neural-network inference in data centres | 256×256 systolic matrix unit of 8-bit MACs with software-managed on-chip memory | Reported about 15–30× higher performance and 30–80× higher TOPS/W than a contemporary CPU and GPU on production workloads | Data-centre scale and fixed function; no RISC-V host, no idle-power or clock-gating study at module level |
| [3] Chen et al. (2017) | Energy dominated by data movement in CNN accelerators | Row-stationary dataflow on 168 PEs in 65 nm CMOS | Silicon-measured energy efficiency from data reuse in the AlexNet convolution layers | Fixed-function CNN chip; power managed by dataflow, not by hardware-detected idle gating; no open RTL or verification flow |
| [7] Genc et al. (2021) | Evaluating DNN accelerators together with their host system | Chisel generator of systolic-array accelerators attached to RISC-V cores, with a software stack | Full-system design-space exploration of accelerator and host | Aimed at design-space exploration; gating and cycle-equivalence of a gated variant are not its focus; large code base |
| [6] Google, Coral NPU | Open RISC-V NPU for edge AI | In-order scalar core, vector unit, and outer-product matrix unit; typically 8×8×4 MAC array; UVM and cocotb tests | Design targets in the guides (for example a 10 mW envelope); whole-core clock gate on WFI or CSR | Targets, not measurements; matrix unit marked experimental in the repository; gating is whole-core, not engine-level |
| [12] Wu et al. (2000) | Registers that hold their value waste clock power | Derive clock-gating conditions from the circuit's own state and activity | Power reduction on sequential benchmark circuits | Gate-level, small sequential circuits; no accelerator, no SoC-level equivalence check |
| [13] Benini and De Micheli (1996) | Idle self-loops in finite-state machines | Synthesis of gated-clock FSMs from an activation function | Power saved by stopping the clock in idle states | Applies to FSM controllers, not to a datapath with operand buffers and accumulators |
| [1] Jacob et al. (2018) | Running neural networks with integer-only arithmetic | int8 weights and activations, int32 accumulation, fixed-point requantization | Accuracy–latency gains for MobileNet inference on mobile CPUs | Software on CPUs; hardware requantization and bit-exact hardware golden models are left to the implementer |

## 2.3 Critical Comparison of Existing Approaches

| Approach | Main Idea | Advantages | Limitations | Evaluation Conditions / Performance |
|---|---|---|---|---|
| Method A<br>Large dataflow and systolic accelerators ([2][3][7]) | Maximise utilization and data reuse with a dedicated array and dataflow | High throughput per watt when the array is busy | Idle time between bursts is not their target; size and complexity exceed a two-person project | ASIC or FPGA, different nodes and workloads; numbers not directly comparable with each other or with this project |
| Method B<br>Open RISC-V NPU ([6]) | Scalar core plus decoupled matrix engine; whole-core gating on WFI or CSR | Complete open system with a software stack; UVM structure to study | Coarse gating granularity; published figures are design targets; very large Chisel code base | Guide-level power targets and design documents; not directly comparable with RTL toggle counts |
| Method C<br>Gate-level clock gating and gated-FSM synthesis ([12] [13]) | Disable the clock of registers whose next state equals their current state | Automatic, well understood, small area overhead | Studied on small sequential circuits; no cycle-equivalence method for a whole SoC | Benchmark circuits at gate level; savings given as percentages of their own baselines. Not directly comparable. |
| Method D<br>Integer-only quantized inference ([1]) | int8 data path with int32 accumulators and fixed-point rescaling | Preserves accuracy; maps well to small MAC arrays | Reference is software; a hardware requantization needs its own golden model | ImageNet MobileNet on mobile CPUs; not directly comparable with an MNIST MLP |

## 2.4 Identified Research Gap

Existing accelerators obtain their energy efficiency mainly from dataflow and utilization [2][3][4]. The idle time of a lightly loaded accelerator that sits behind a scalar core is handled coarsely, by gating the whole core [6], or only at gate level on small circuits [12][13]. Within the sources reviewed here, none combines three elements that this project needs together: (a) module-level, hardware-detected-idle gating of a small RISC-V-attached int8 engine; (b) a cycle-identical D0/D1 equivalence criterion that is checked by a retire-trace hash rather than assumed; and (c) a measured relation between switching-activity reduction and the engine's idle ratio on both synthetic and real (MNIST) workloads against a bit-exact golden model.

This is a gap in evidence, not a claim of a new gating circuit. The latch-plus-AND clock gate is textbook [14]. Switching-activity counts are also not power: they ignore capacitance, glitching, and the gate's own overhead [15], so the project reports them as estimates and separates them from the functional evidence.

## 2.5 Positioning Statement

Compared with the reviewed approaches, this project does not propose a new circuit-level technique [12][13] or a larger accelerator [2][7]. Its contribution is a complete, reproducible, small-scale study: an NPU designed and verified from the instruction set down to RTL, gated at the module level, and shown by evidence to be equivalent to its ungated twin. It differs from [6] by gating the MAC engine on hardware-detected idle instead of the whole core on a software signal, and by building its own 5-stage core, golden models, and UVM environments so that every design decision can be explained. Evaluation uses QuestaSim RTL simulation, sweeping the idle ratio (10, 25, 50, 75, and 90 %) and measuring toggles per 1,000 cycles, compared with the ungated design as ground truth.

# 3 PROPOSED IDEA / PROPOSED SOLUTION

*Describe the solution concept, architecture, novelty, and testable hypotheses.*

## 3.1 Core Technical Idea

This project introduces a CoralNPU-inspired RISC-V NPU in which four custom-0 instructions push commands through an 8-entry FIFO to a decoupled int8 outer-product MAC engine (4×4×4 = 64 MACs per cycle), and in which the clock of the engine's datapath (operand buffers, PE array, accumulators) is gated by a latch-plus-AND cell. The enable is asserted whenever the engine's FSM is not idle, its FIFO is not empty, or reset is active. Because the enable depends only on registered state, a command pushed at cycle t opens the gate in time for the first edge the datapath needs, at cycle t+2, so gating adds no cycle and D1 can be cycle-identical to D0. The design is verified by self-built UVM environments and a bit-exact MNIST MLP, and the benefit of gating is quantified as toggle counts versus the engine's idle ratio. This addresses the gap of Section 2.4: gating at the engine level, equivalence by evidence, and activity measured on real and synthetic workloads.

## 3.2 Proposed System Architecture

| Architecture Element | Description / Interface / Function |
|---|---|
| Input / Data acquisition | Bare-metal firmware (C and assembly, custom instructions through `.insn` wrappers) and the int8 MLP data loaded into ITCM and DTCM by backdoor `$readmemh` before reset. MNIST pixels p are quantized as (p·254 + 255) // 510 to 0..127; the 784-64-32-10 model is trained in a notebook and exported as `.mem` files with SHA-256 hashes. |
| Pre-processing / Interface layer | 5-stage RV32IM + Zicsr + Zifencei + Zbb core decodes the custom-0 instructions and pushes LOAD_WEIGHT, LOAD_INPUT, and COMPUTE commands from the MEM stage into the 8-entry command FIFO; READ_RESULT stalls until the engine has drained. |
| Proposed core module | `mac_engine`: FSM (IDLE, CHECK, LOAD, MAC); operand buffers WBUF and IBUF (256 × 128 bit); 16 dot4 PEs and a 4×4 int32 accumulator tile (wrap modulo $2^{32}$); CSRs npu_status and npu_busy_cyc. Operands come from DTCM port B (128 bit, read-only). |
| Optimization / Control / Scheduling | `npu_clk_gate` (latch transparent while clk is low, then AND) produces clk_mac. Enable: !rst_n \|\| (fsm != IDLE) \|\| !fifo_empty. D0 ties the enable to 1 (GATING_EN = 0); D1 uses it (GATING_EN = 1); the two builds differ in nothing else. |
| Output / Decision / Actuation | READ_RESULT returns accumulator entries to a scalar register; firmware adds the bias, requantizes with clamp((acc·M + 2^(S−1)) >> S, 0, 127), and takes the argmax of the first 10 of 12 padded logits. The test ends with a write to SIM_CTRL. |
| Monitoring / Evaluation interface | Three UVM environments (mac_env, core_env, soc_env) with C golden models over DPI-C (npu_ref, an RV32 instruction-set simulator, and a Python integer model), SystemVerilog assertions, functional coverage, a passive retire-trace agent, the D0/D1 equivalence harness, and a windowed VCD dump of the engine for toggle counting. |

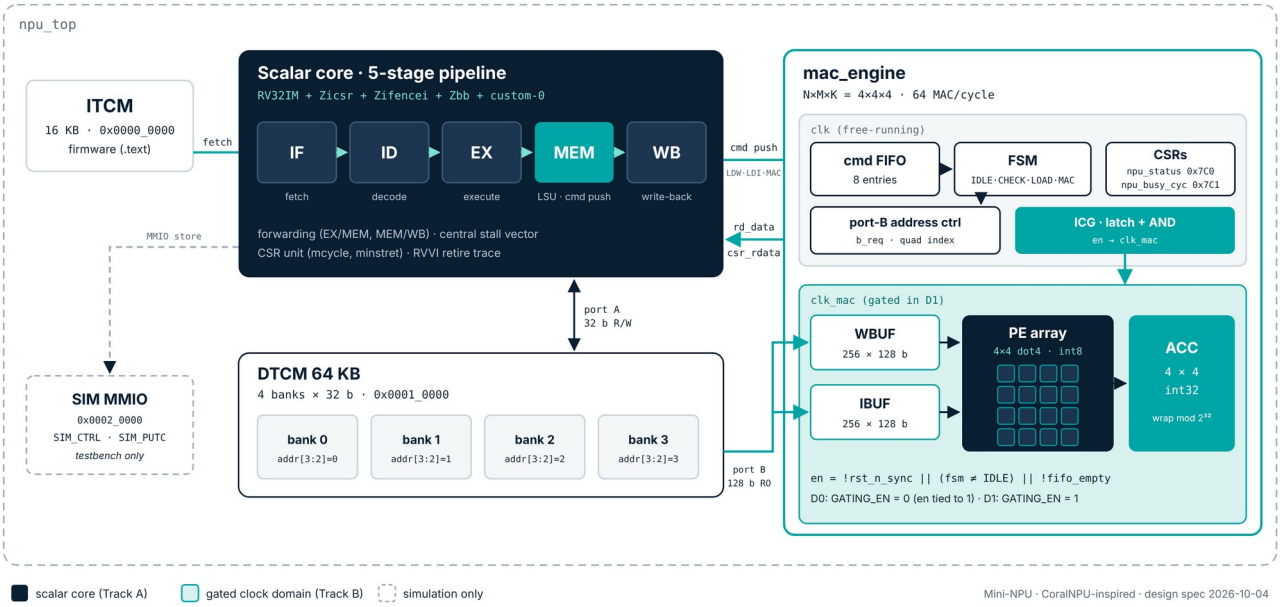



*Figure 1. Mini-NPU system architecture (npu_top). The scalar core (dark) and the decoupled MAC engine, whose datapath runs on the gated clock clk_mac (teal), share a 64 KB dual-port DTCM. The SIM MMIO block exists in simulation only.*

## 3.3 Novelty and Expected Technical Contributions

**Contribution 1.** A working small-scale RISC-V NPU in RTL: a custom-0 instruction extension, a command FIFO, a decoupled int8 outer-product engine, and a dual-port DTCM, running an MNIST MLP bit-exactly against a golden model.

**Contribution 2.** A module-level clock-gating scheme with an evidence-based equivalence method: retire-trace SHA-256, final mcycle and minstret, and output dumps compared between D0 and D1 for every soc-level test and every MNIST run.

**Contribution 3.** A measured relation between switching-activity reduction (M1 and M2) and the engine's idle ratio, cross-checked on MNIST against the synthetic curve.

**Contribution 4.** A self-built UVM verification infrastructure (three environments, DPI-C golden models, assertions, and coverage) and a reproducibility package that regenerates the results with one command per flow.

*The novelty lies in the combination and in the evidence, not in a new gating circuit (Section 2.4).*

## 3.4 Hypotheses

| Hypothesis | Testable Statement | Evidence Required |
|---|---|---|
| H1 | The gated design D1 produces the same retire-trace hash, final mcycle and minstret, and output dump as D0 for every soc-level test, the synthetic workloads, and the MNIST runs. | Equivalence harness: hash and counter comparison per test, with identical binary, seed, and simulator |
| H2 | The reduction of M1 (toggles on clk_mac, per 1,000 cycles) grows with the engine's idle ratio and approaches the idle fraction, whereas the reduction of M2 (all engine nets) is smaller and depends on data activity. | E2 curve over idle ratios of 10, 25, 50, 75, and 90 %, D1 versus D0 |
| H3 | The MNIST workload, placed at its own idle ratio (npu_busy_cyc / mcycle), falls within the spread of the synthetic E2 points. | E4 toggle counts for 100 MNIST images compared with the E2 curve |
| H4 | The self-built UVM environments close 100 % of the declared coverage bins on D0 with zero scoreboard mismatches and zero assertion failures. | Merged coverage report; regression logs with UVM_ERROR and UVM_FATAL counts |

### 3.5 Scope

**IN SCOPE:**

- The core extensions M, Zicsr, Zifencei, and Zbb, and the four custom instructions with their command FIFO.
- The MAC engine (FIFO, operand buffers, PE array, accumulators, CSRs), the dual-port DTCM, the ITCM, and the simulation MMIO.
- The clock-gating cell and the D1 configuration; the D0/D1 equivalence harness.
- Three UVM environments, a C golden model of the engine, a C instruction-set simulator, and an integer golden model of the MLP.
- Bare-metal firmware and the MNIST pipeline; the toggle-counting flow and its figures.

**OUT OF SCOPE / NON-GOALS** (deferred to the graduation thesis):

- RVV/Zve32x and the F extension; DMA, multiple accumulator tiles, and a pipelined PE.
- Synthesis, place-and-route, FPGA bring-up, and any absolute or gate-level power claim; all power numbers are RTL switching-activity estimates.
- A compiler toolchain (IREE or TensorFlow Lite Micro [19]) and frontdoor (bus) loading of programs.
- LLM-assisted verification. The 8×8 array is built only if time remains after the core criteria are met.

# 4 METHODOLOGY / IMPLEMENTATION AND EVALUATION PLAN

*Explain how the proposed idea will be built, tested, compared, and analyzed.*

## 4.1 Overall Research Methodology

| Phase | Main Activities | Output / Evidence |
|---|---|---|
| A. Problem and baseline definition | Fix the scope; freeze the custom ISA and the engine interface; check the tools (UVM, DPI-C, coverage); write the golden model of the engine with unit tests. | Design spec with decisions D1–D24; frozen interface; tool-gate results |
| B. Design | Extend the core (M, Zicsr, Zifencei, Zbb, custom-0, retire trace); write the dual-port DTCM, the MAC engine D0, and the SoC top; write the firmware base. | RTL; block notes; directed-test and regression results |
| C. Implementation and verification | Build mac_env, core_env, and soc_env; write constrained-random sequences (200 or more seeds), assertions, and coverage; add the clock gate (D1) and the equivalence harness. | Regression logs; merged coverage report; D0/D1 equivalence table (SC1–SC3) |
| D. Experimentation | Run the synthetic idle-ratio sweep and the MNIST runs on D0 and D1; dump VCD only inside the measurement window. | Toggle-count CSV files; MNIST logits compared with the golden model (SC4, SC5) |
| E. Analysis | Compute the percentage change of M1 and M2; cross-check MNIST against the synthetic curve; document limitations; write the report and reproducibility package. | Figures and tables; English report; slides; one-command reproduction (SC6) |

## 4.2 Implementation Plan

| Step | Input | Processing / Design Action | Output |
|---|---|---|---|
| 1 | CoralNPU documentation; course requirements | Study the open design; write the spec, decision log, and success criteria SC1–SC6 | Approved spec; frozen ISA and interface |
| 2 | Spec sections 6, 7, 9 | Write the golden models: npu_ref (C), an RV32 instruction-set simulator (C), and an integer MLP model (Python); gate each with its own unit tests | Oracles that pass their unit tests |
| 3 | Existing 5-stage RV32I core | Add M, Zicsr, Zifencei, Zbb, custom-0 decode and push, ITCM and DTCM, and the retire-trace port; check against a Python reference ISS | Core regression green |
| 4 | Frozen engine interface | Write the command FIFO, FSM, LOAD and MAC states, PE array, CSRs, and reset; measure latency per command | MAC engine D0; directed suite green |
| 5 | Core and engine RTL | Integrate them in the SoC top; write firmware tests and synthetic and MNIST programs | SoC smoke tests, 16 of 16 on D0 and D1 so far |
| 6 | Directed and random plans; coverage plan | Build the UVM environments; random sequences over 200 or more seeds; assertions A-MAC-1..5; close coverage on D0 | SC1 and SC2 evidence |
| 7 | D0 design | Add the clock-gate cell and the enable logic; run every soc test and every MNIST run on D0 and D1; compare retire-trace hash, mcycle, minstret, and logits | SC3 equivalence table |
| 8 | Equivalent D0 and D1 | Run the synthetic sweep and the MNIST toggle run; count toggles inside the measurement window | SC4 data, plots, and cross-check |
| 9 | All flows | Make each flow runnable by one command; record tool versions, seeds, and the git hash; test on a clean clone | SC6 reproducibility package |

## 4.3 Hardware and Software Platform

| Component / Tool | Version / Specification | Purpose in the Project |
|---|---|---|
| Processing platform | Windows PC with WSL2 (Ubuntu 24.04) | Host for QuestaSim and for the Linux tools of the regression |
| Sensors / Interfaces | Not applicable (simulation only) | No board bring-up in this project |
| Operating system / RTOS | Windows with WSL2 Ubuntu 24.04; CI on Ubuntu 24.04. No RTOS: bare-metal firmware | Same Linux environment locally and in continuous integration |
| Programming language | SystemVerilog and Verilog-2001 (RTL, testbenches); C99 (golden models, DPI-C); Python 3; RISC-V assembly and C (firmware, `-march=rv32im_zicsr_zifencei_zbb`) | RTL and verification; golden models and firmware; training, data export, and toggle counting |
| Framework / Library | UVM 1.2 (IEEE 1800.2 [16]); numpy; PyTorch (training only); pytest | Verification methodology; integer reference and its unit tests; MLP training |
| Development / Simulation tool | QuestaSim 10.7c; Verilator 5.020 (lint); Icarus Verilog 12 (core regression); RISC-V GNU toolchain; GitHub Actions | Simulation, coverage, and VCD dumps; lint; regression of the core; compilation of firmware; automatic regression on every push |

## 4.4 Data, Workloads, Benchmarks, or Test Scenarios

| Item | Definition |
|---|---|
| Dataset / benchmark / workload | (1) MNIST [9] with a 784-64-32-10 MLP quantized to int8. (2) Synthetic firmware that alternates NPU bursts (LOAD_WEIGHT, LOAD_INPUT, and COMPUTE with K = 256) with scalar busy loops. (3) Directed and constrained-random tests for the engine, the core, and the SoC. |
| Size / number of samples / duration | MNIST: 100 images (25 batches of 4) for the activity run, the first 1,000 test images for the bit-exact comparison on D0 and D1, and the 10,000-image test set once on D0. Synthetic: five idle ratios (10, 25, 50, 75, 90 %). Random tests: 200 or more seeds per environment. |
| Pre-processing / normalization / compilation | Pixels to int8 by (p·254 + 255) // 510. Symmetric int8 weights in ±127, int32 biases. Per-layer requantization with an integer multiplier M and shift S chosen as the largest S for which bound·M + 2^(S−1) ≤ $2^{31}-1$. The last layer is padded from 10 to 12 outputs; argmax ignores the padding. |
| Train / validation / test split | Training on the 60,000-image training set (seed 0); 10,000 training images calibrate the activation scales (99.99th percentile); evaluation on the 10,000-image test set. Hardware comparisons use test images only. |
| Operating conditions / input ranges | RTL simulation of one clock domain, no timing back-annotation. D0 and D1 use the same binary, the same simulator version, and the same dump options; random seeds are explicit. |
| Ground truth / reference output | Independent golden models (engine, instruction-set simulator, integer MLP) and the retire trace of the core; the models are used only after their own unit tests pass. |

## 4.5 Baselines and Comparative Methods

| Baseline | Reason for Selection | Configuration / Fairness Controls |
|---|---|---|
| Baseline 1: D0 (GATING_EN = 0) | Same hierarchy and netlist as D1 apart from the enable tie; the reference for equivalence and for switching activity | Same binary, seed, simulator version, and dump window; each configuration run once, reruns only after a logged external failure |
| Baseline 2: independent golden models | Oracle for functional correctness; bit-exact comparison removes tolerance arguments | Used only after their own unit tests pass; cross-checked against one another (the integer MLP model against the engine model, the instruction-set simulator against a Python reference ISS) |
| Baseline 3: float MLP | Accuracy reference for the int8 model [1] | Same trained weights, same 10,000 test images |

## 4.6 Evaluation Metrics

| Dimension | Candidate Metrics | Selected Metric(s) and Measurement Method |
|---|---|---|
| Quality / ML | Accuracy, label agreement | int8 and float test accuracy; logits and labels bit-exact against the golden model |
| Performance | Cycles, latency, throughput | mcycle and minstret; busy cycles per engine command (bound K/4 + 3); idle ratio = npu_busy_cyc / mcycle; D0 and D1 must match exactly |
| Compute / Memory | MACs per cycle, memory footprint | Peak 64 MACs per cycle; DTCM data region 56,816 B of 65,536 B |
| Hardware | LUT, FF, area, frequency | Not evaluated (no synthesis) |
| Energy | Power, energy per inference | Switching-activity estimate: M1 = toggles on clk_mac; M2 = sum of toggles over all engine nets, split into free-clock and gated-clock parts; both per 1,000 cycles |
| Reliability | Mismatches, assertion failures, coverage | Scoreboard mismatches and assertion failures (target 0); merged functional coverage (target 100 % of declared bins) |

## 4.7 Experimental Design

| Experiment | Objective | Independent Variables | Measured Outputs | Decision / Comparison |
|---|---|---|---|---|
| E1: Functional verification | Show that D0 and D1 are correct against the golden models | Test (directed, random seed); configuration D0 or D1 | Scoreboard mismatches; assertion failures; coverage % | Accepted only with 0 mismatches, 0 failures, and 100 % of declared bins (SC1, SC2) |
| E2: Synthetic activity sweep | Relate switching activity to the idle ratio | Idle ratio 10, 25, 50, 75, 90 %; configuration | M1 and M2 per 1,000 cycles; % change D1 versus D0 | Report the curve; test H2; run only after E3 passes |
| E3: D0/D1 equivalence | Show that gating changes neither timing nor results | Workload (every soc test, synthetic, MNIST); configuration | Retire-log SHA-256; mcycle; minstret; logits dump | Any difference is a failure (SC3) |
| E4: MNIST | Show bit-exact inference and measure its activity | Image count (100, 1,000, 10,000); configuration | Logits and labels versus golden; accuracy; M1, M2; idle ratio | Bit-exact (SC5); place the result on the E2 curve (H3) |

## 4.8 Analysis and Reproducibility Plan

- Every simulation runs through one Python runner (options for the simulator, the test target, the configuration D0 or D1, the seeds, and coverage); logs are kept per run under a dated directory with a `manifest.yaml` that records QuestaSim version and licence type, Verilator, GCC, and Python versions, the git hash, seeds, and the SHA-256 of each log.
- Random seeds are explicit (for example 1..200). Golden models are gated: each passes its own unit tests before it is used as an oracle, and testbenches are checked by injecting faults and confirming that the checks fail.
- Activity dumps are taken only inside a firmware-controlled window (MEAS_START and MEAS_STOP) and only for the engine scope. A script converts the VCD to CSV and a second script draws the figures. Every figure states configuration, workload, seed, and unit and is labelled “switching-activity estimate”.
- Functional evidence (UVM) and power evidence (toggles) are reported in separate sections. No D1-versus-D0 activity figure is produced before the equivalence criterion (SC3) is met.
- One command per flow regenerates the regression, the coverage, and the activity data; this is checked on a clean clone in week 15 (SC6).

## 4.9 Technical Risks and Mitigation

| Risk | Impact | Mitigation / Contingency |
|---|---|---|
| Simulator licence lacks a feature (UVM, randomize, covergroup, DPI-C) | UVM flow blocked | Checked in week 1: QuestaSim 10.7c ran all four. Fallbacks: Vivado xsim (verified then), Verilator, or pyuvm. |
| Simulation too slow for the MNIST batches | SC5 or E4 cannot finish | Measure in week 14. Cut order: the 10,000-image run first, then E4 down to 20 images; SC3 on MNIST and the report are never cut. Long runs may move to Verilator with a QuestaSim cross-check. |
| Gating changes timing (not cycle-identical) | SC3 fails | By construction the first gated edge arrives when the datapath needs it; assertion A-GATE-1 forbids gating pending work; fallback is result-identical, with the reason documented. |
| Interface changes between the two tracks | Rework in both | Interface frozen at week 2 (both students signed on 2026-10-29); later changes need both signatures and a spec update. |
| Golden model and RTL share a misreading of the spec | Wrong results accepted | Independent models in C and Python cross-checked against one another; testbench fault-injection; hand-computed unit cases. |
| Firmware overwrites DTCM data that a queued LOAD still reads | Corrupt inputs | Software rule (drain with READ_RESULT before overwriting) plus an assertion and a negative test that proves it fires. |
| Schedule pressure | Late deliverables | Priority order: D0 correct, then D1 equivalent, then switching activity, then the MNIST demo; the 8×8 array and PE pipelining are dropped first. |

# 5 EXPECTED RESULTS

*Define measurable outcomes, expected contributions, and concrete project deliverables.*

## 5.1 Expected Technical Results

The verified D0 design is expected to pass its full regression with zero scoreboard mismatches and zero assertion failures, and to close 100 % of the declared functional-coverage bins. The gated design D1 is expected to be cycle-identical to D0 on every soc-level test and MNIST run, which is the condition for reporting any activity figure. The 784-64-32-10 int8 MLP is expected to produce logits and labels that match the golden model bit-exactly on the first 1,000 test images, and to reproduce the golden accuracy on the full 10,000-image run. For switching activity, M1 is expected to fall roughly in proportion to the idle fraction, so that a high idle ratio removes most clk_mac toggles, while M2 is expected to fall less and to depend on data activity. No target percentage is fixed in advance. The MNIST point is expected to lie within the spread of the synthetic curve at its own idle ratio.

## 5.2 Target Performance Table

| Metric | Baseline / Current Value | Expected Target | Measurement Condition |
|---|---|---|---|
| Primary metric: D0/D1 equivalence | D0 only: clk_mac toggles in every cycle. Smoke programs on D0 and D1 are cycle-identical (16 of 16, 2026-11-10). | Identical retire-trace hash, mcycle, minstret, and outputs for 100 % of soc tests and MNIST runs | Same binary, seed, simulator; E3 |
| Secondary metric: switching activity | D0 toggles per 1,000 cycles at each idle ratio (to be measured) | % change of M1 and M2 reported at idle ratios 10–90 % and for MNIST; no preset target | VCD window MEAS_START to MEAS_STOP, engine scope; E2 and E4 |
| Functional metric: mismatches and coverage | mac_env on D0 and D1: 224 of 224 runs as expected, 81,332 scoreboard comparisons, 0 mismatches, 100 % of 7 covergroups (project log, 2026-11-10) | Same result for core_env and soc_env; 0 mismatches and 100 % of declared bins | QuestaSim 10.7c, UVM 1.2, runner with `--cov`; E1 |
| MNIST metric | int8 test accuracy 97.43 % (float 97.48 %) on 10,000 images | RTL logits and labels bit-exact on the first 1,000 images (D0, D1); full run reproduces the golden accuracy | Golden dump compared with the RTL LOGITS dump; E4 |
| Reproducibility metric | Per-run manifest exists; one command for the engine regression | One command per flow, verified on a clean clone | Week 15; SC6 |

## 5.3 Expected Scientific / Engineering Contributions

- A small, fully explained RISC-V NPU (custom-instruction extension, decoupled int8 engine, dual-port DTCM) that runs a quantized MNIST MLP bit-exactly.
- An equivalence method for clock gating, demonstrated at module level by retire-trace hash and counter comparison, rather than assumed from the circuit structure.
- A measured relation between the idle ratio of an NPU engine and the switching activity removed by gating, with a real-workload cross-check.
- A reusable, self-built UVM environment with golden models, assertions, coverage, and a reproducibility package for later work such as the graduation thesis.

### 5.4 Expected Deliverables

| Deliverable | Minimum Content / Acceptance Evidence |
|---|---|
| Research proposal | Approved problem, gap, idea, methodology, and expected results |
| Literature review | Critical synthesis of the related work (Section 2) and the related-work part of the final report |
| Design artifacts | Design spec with its decision log, block notes for the core and the MAC engine, the architecture figure, and the interface definitions |
| Prototype | RTL of the core, MAC engine, dual-port DTCM, and SoC top in D0 and D1; firmware; golden models |
| Experiment package | UVM environments, test lists, the runner, logs with manifests, coverage reports, and toggle-count CSV files |
| Technical report | English report with verification, switching-activity, and MNIST chapters, discussion of limitations, and conclusion |
| Final presentation and demo | Live regression, D0/D1 equivalence, MNIST inference, and the activity plots |
| Repository | GitHub repository with RTL, testbenches, models, firmware, scripts, documents, and the reproducibility package |

### 5.5 Interpretation of Negative or Mixed Results

- If D1 is not cycle-identical to D0, the project falls back to result-identical behaviour and documents where the cycles are added and why. The study then shows the cost of gating instead of its absence of cost.
- If the gating benefit is small at low idle ratios, this agrees with the model that gating cannot help a busy engine. The measured curve then shows the break-even point, and a low MNIST idle ratio shows that this workload keeps the engine busy and that module-level gating is the wrong tool for it.
- If M2 falls much less than M1, the data activity inside the engine dominates, which points to operand-level techniques such as operand isolation as future work.
- All activity figures come from RTL simulation of one design and one simulator. They are evidence about this design, not claims about silicon power or about NPUs in general.

## Project Timeline and Milestones

*Dates follow the project calendar: week 1 starts Monday 12 October 2026 and week 16 ends Sunday 31 January 2027. Status is given as of 11 November 2026 (week 5).*

| Week | Activities | Milestone / Deliverable (status 2026-11-11) |
|---|---|---|
| 1<br>12–18 Oct | Tool gate: QuestaSim UVM, DPI-C, coverage, VCD scope, toolchain; problem scoping | Tool-gate results. *Done 2026-10-18.* |
| 2<br>19–25 Oct | ISA and CSR documents; engine interface; engine golden model with unit tests | Frozen ISA and interface. *Golden model done 2026-10-17; interface signed by both on 2026-10-29.* |
| 3–4<br>26 Oct–8 Nov | M, Zicsr, Zifencei, Zbb, custom-0 decode and push; instruction-set simulator; MAC engine D0; dual-port DTCM | Phase 1 exit. *Core regression 1,291 of 1,291; engine D0 and D1 green on QuestaSim (2026-11-05); one core policy decision (W2-1) still open.* |
| 5–6<br>9–22 Nov | SoC integration and firmware base; mac_env and core_env directed tests | First soc test. *Engine-side integration done (2026-11-10); mac_env directed and random done (2026-11-07).* |
| 7–9<br>23 Nov–13 Dec | Random tests and coverage; soc tests; coverage closure | SC1 and SC2 on D0; verification chapter draft. *mac_env: 100 % of 7 covergroups (2026-11-10); core_env and soc_env pending.* |

| Week | Activities | Milestone / Deliverable (status 2026-11-11) |
|---|---|---|
| 10–11<br>14–27 Dec | Clock-gate cell, D1 regression, equivalence harness | SC3 on every soc test |
| 12<br>28 Dec–3 Jan | Synthetic firmware; toggle-counting script; E2 runs | E2 data and plots (SC4, synthetic part) |
| 13–14<br>4–17 Jan | MNIST firmware; weight packing and golden files; MNIST 1,000 images on D0 and D1; report skeleton | SC5 (bit-exact on D0 and D1) |
| 15<br>18–24 Jan | E4 MNIST toggle run (100 images); full 10,000-image run on D0; reproduction on a clean clone | SC4 (MNIST), SC6 |
| 16<br>25–31 Jan | English report, slides, reproducibility package; final validation and demo | Final report, demo, and repository |

## References


[1] B. Jacob *et al.*, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, Salt Lake City, UT, USA, 2018, pp. 2704–2713.

[2] N. P. Jouppi *et al.*, "In-datacenter performance analysis of a Tensor Processing Unit," in *Proc. 44th Annu. Int. Symp. Computer Architecture (ISCA)*, Toronto, ON, Canada, 2017, pp. 1–12.

[3] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, "Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks," *IEEE J. Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, Jan. 2017.

[4] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, "Efficient processing of deep neural networks: A tutorial and survey," *Proc. IEEE*, vol. 105, no. 12, pp. 2295–2329, Dec. 2017.

[5] M. Horowitz, "Computing's energy problem (and what we can do about it)," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, 2014, pp. 10–14.

[6] Google, "Coral NPU," GitHub repository *google-coral/coralnpu*, commit 382b5c1, and Coral NPU developer guides (developers.google.com/coral/guides), read Oct. 15, 2026.

[7] H. Genc *et al.*, "Gemmini: Enabling systematic deep-learning architecture evaluation via full-stack integration," in *Proc. 58th ACM/IEEE Design Automation Conf. (DAC)*, San Francisco, CA, USA, 2021, pp. 769–774.

[8] A. Waterman and K. Asanović, Eds., *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*, Document Version 20191213, RISC-V Foundation, Dec. 2019.

[9] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.

[10] J. L. Hennessy and D. A. Patterson, "A new golden age for computer architecture," *Commun. ACM*, vol. 62, no. 2, pp. 48–60, Feb. 2019.

[11] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design RISC-V Edition: The Hardware/Software Interface*, 2nd ed. Morgan Kaufmann, 2021.

[12] Q. Wu, M. Pedram, and X. Wu, "Clock-gating and its application to low power design of sequential circuits," *IEEE Trans. Circuits Syst. I, Fundam. Theory Appl.*, vol. 47, no. 3, pp. 415–420, Mar. 2000.

[13] L. Benini and G. De Micheli, "Automatic synthesis of low-power gated-clock finite-state machines," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 15, no. 6, pp. 630–643, Jun. 1996.

[14] M. Keating, D. Flynn, R. Aitken, A. Gibbons, and K. Shi, *Low Power Methodology Manual: For System-on-Chip Design*. New York, NY, USA: Springer, 2007.

[15] F. N. Najm, “A survey of power estimation techniques in VLSI circuits,” *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 2, no. 4, pp. 446–455, Dec. 1994.

[16] *IEEE Standard for Universal Verification Methodology Language Reference Manual*, IEEE Std 1800.2-2017, 2017.

[17] *IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language*, IEEE Std 1800-2017, 2018.

[18] W. Stallings, *Computer Organization and Architecture: Designing for Performance*, 11th ed. Pearson, 2019.

[19] R. David *et al.*, “TensorFlow Lite Micro: Embedded machine learning for TinyML systems,” in *Proc. Mach. Learn. Syst. (MLSys)*, vol. 3, 2021, pp. 800–811.